

Code Ko Git & Cloud Par Deploy Karne Ki Complete Process

Complete Reference Notes: Git Multi-Branch Push, GitHub Pages Setup, Fixes & Render Backend Deployment

PART 1: Code Ko Git Me Push Krne Ke 2 Method

1ST METHOD Pura Code Backend Aur Frontend Dono Ko 1 Hi Branch Main

backend + frontend to normal hai pura code likh do aur commit kr do:

```
git add .  
git commit -m "Initial commit: backend and frontend code"  
git push origin main
```

Iske hisab se deploy karein toh:

1. Select GitHub repo
2. Then click on **Settings**
3. Then click on **Pages**
4. Then select branch: **main** and select **/root**
5. Then **Save**

⚠ **Problem: GitHub pr deploy ho jayegi lekin deployment ke baad jo URL milega vo nahi chalega!**

Jab aapne poora project (frontend aur backend) ek hi branch mein daal kar GitHub Pages par deploy kiya, toh woh isliye fail hua kyunki GitHub Pages sirf static websites (HTML, CSS, UI) host karne ke liye bana hai:

- **No Server Environment:** GitHub Pages ke paas Node.js, Express, ya database (MongoDB) chalane ki power (runtime) nahi hai. Woh aapke backend code ko execute nahi kar sakta; woh unhe bas normal text files ki tarah read karta hai.
- **Missing index.html:** GitHub Pages hamesha root (sabse bahar wale folder) mein ek `index.html` file dhoondhta hai jisse website start ho. Jab aapne dono folders (client aur server) ek sath push kiye, toh use root mein `index.html` nahi mila (woh aapke client ya dist folder ke andar chhipa tha). Isliye screen par error aaya ya kuch nahi dikha.

⚠ **Error (ERR_CONNECTION_REFUSED) Fix**

Yeh error (ERR_CONNECTION_REFUSED) isliye aati hai kyunki aapka live frontend (GitHub Pages par) abhi bhi aapke local computer (localhost:5000) se data maang raha hai. Ise hamesha ke liye fix karne ke steps yeh hain:

Step 1: Backend mein CORS Update Karein

Apne backend code (Express server) mein cors origin ko apne live GitHub Pages URL par set karein. Code ko is tarah update karein:

```
app.use(cors({
  origin: 'https://himanshu6666.github.io', // (yaha pr jo github m deployment ke
  baad jo url mila jo chal nahi raha usse paste krna hai)
  credentials: true
}));
```

Step 1: Vite Config Update Karna

Apne client folder ke andar vite.config.js file open karein. defineConfig ke andar base property add karein. Yeh aapke GitHub repository ka exact naam hona chahiye (aage aur peeche slash ke sath):

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  base: '/DevConnect/', // (folder ka naam jo main folder hai jiske andar client ya
  server folder h)
})
```

Step 2: gh-pages Package Install Karna

Terminal open karein aur apne client folder ke andar jayein:

```
cd client
npm install gh-pages --save-dev
```

Step 3: package.json Update Karna

client folder ki package.json file open karein.

scripts section ke andar predeploy aur deploy commands add karein:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "oxlint",
  "preview": "vite preview",
  "predeploy": "npm run build",
  "deploy": "gh-pages -d dist"
}
```

Step 4: Deploy Command Chalana

Terminal (client folder ke andar) mein yeh final command chalayein:

```
npm run deploy
```

Yeh command pehle predeploy chalayegi jisse dist folder generate hoga. Phir yeh gh-pages tool ko use karke us dist folder ke saare data ko aapke GitHub par ek nayi gh-pages naam ki branch banakar push kar degi.

Then backend code commit aur push karein:

```
git add .
git commit -m "update cors origin"
git push origin main # (backend branch name)
```

Step 5: GitHub Pages Settings Configure Karna

1. Apne GitHub account par **DevConnect** repository kholien.
2. Top menu se **Settings** par click karein, aur left sidebar se **Pages** select karein.
3. Source dropdown ko "**Deploy from a branch**" par set karein.
4. Branch dropdown menu mein **gh-pages** select karein.
5. Folder dropdown ko **/ (root)** hi rehne dein aur **Save** par click kar dein.

✓ GitHub Action background mein run hoga aur 1-2 minute baad jab aap apna himanshu6666.github.io/DevConnect/ URL open karenge, toh aapka Vite frontend bilkul sahi load ho jayega (jisse ki code ka frontend wala part chal jayega aur website m show krega).

2ND METHOD Frontend Ki Alg Branch Aur Backend Ki Alg Branch

Backend Branch Push Karna

Pahle ham git add (folder ka naam jisme backend hai jo bhi folder hai) karenge:

```
# For ex:
git add server/

# Then:
git commit -m "Backend:Added backend file"

# Git push origin (branch name):
git push origin main
```

Frontend Branch Push Karna

Then again client folder ko add karenge:

```
# Then gain:
git add client/

# Commit:
git commit -m "frontend:-added frontend file"
```

```
# Git push origin (frontend branch name):  
git push origin frontend
```

GitHub Pages Deployment Steps

1. Open GitHub
2. Select repository
3. Click on **Settings**
4. Click on **Pages**
5. Then git to for deployment select "**from branch**" and then select **frontend** branch and select **root** and then **Save**.

✓ Then git m deploy ho jayega.

⚠ Important Note:

Git m keval frontend chalta hai backend nahi.

Backend ko chlane ke liye hame website ko cloud ya **Vercel ya Render etc** m deploy krna hoga.

Then ham deploy krna shikenge.

PART 2: Render M Deploy Krne Ke Liye Process

STEP 1 Render Par Naya Project Banayein

- **Render.com** par login karein.
- Dashboard mein top-right par **New +** par click karein aur **Web Service** chunein.
- "**Build and deploy from a Git repository**" select karke apna **DevConnect** GitHub repo connect karein.

STEP 2 Sahi Branch aur Folder Select Karein

Yahan apne project structure ke hisaab se details bharein:

- **Name:** API ka koi bhi naam dein (jaise devconnect-api).
- **Branch:** Jis branch mein aapka backend code hai (jaise main ya backend-branch).
- **Root Directory:**
 - Agar backend code server naam ke folder ke andar hai, toh yahan server likhein.
 - Agar backend code bina kisi folder ke seedha branch ke bahar rakha hai, toh ise blank (khali) chhod dein.

STEP 3 Build aur Start Commands Daalein

- **Runtime:** Node select karein.
- **Build Command:** `npm install` type karein.
- **Start Command:** `node src/server.js` (ya jo command aapka server locally start karti hai) ex - jaise hamare m folder server ke andr folder src m server.js hai to) type karein.

STEP 4 Environment Variables (.env) Setup Karein

- Page scroll karke **Environment Variables** section par aayein.
- **"Add Environment Variable"** par click karke apne local `.env` file ka saara zaroori data yahan daalein (jaise `MONGO_URI`, `JWT_SECRET`). (Note: `PORT` variable add karne ki zarurat nahi hai, Render khud assign kar leta hai).

.env m pura data copy krke dal dena hai

ex - `mongodburl=123`

to environment m **mongoduri**

aur value m **123**

esse hi `.env` file ka pura data dlina hai port ko chodkr

STEP 5 Deploy Karein aur Live URL Copy Karein

- Sabse niche **Create Web Service** par click karein.
- Terminal logs aana shuru honge. 2-3 minute wait karein jab tak green color mein **"Live"** status na dikhe.
- Screen ke top-left par ek live URL milega (jaise `https://devconnect-api.onrender.com`). Is URL ko copy kar lein.

STEP 6 Frontend Mein URL Update Karke Re-deploy Karein

- Apne local code editor mein frontend (client) ka code open karein.
- Apni API settings (jaise `api.js`, Axios base URL, fetch requests, ya `.env` file) mein `http://localhost:5000/api` ko hata kar yeh naya Render live URL route ke sath paste kar dein:

```
// Axios Base URL ya API Config me Render URL ke sath /api jod dein:  
baseURL: 'https://devconnect-api.onrender.com/api'
```

- Terminal mein client folder ke andar jayein aur apni pehle wali deployment command chalayein:

```
cd client  
npm run deploy
```

✓ Yeh updated frontend code ko GitHub Pages par push kar dega, aur aapka UI live backend se connect ho jayega.

Production Deployment & Routing Optimization Notes

Technical Reference & Assignment Guide: Fixing GitHub Pages Routing, Centralized Axios API, and Razorpay Gateway

React Router DOM

GitHub Pages (gh-pages)

Axios Instance

Razorpay Integration

Render Backend

1. GitHub Pages Routing & Default Marketplace View

Fixing sub-path routing and initial empty view on deployment

Issue Encountered: GitHub Pages repository URL load hone par sirf Navbar show ho raha tha aur user ko Marketplace manual click karke open karna pad raha tha. Saath hi GitHub repo sub-directory path hone ki wajah se direct routes mismatch ho rahe the.

Solution Approach:

1. `BrowserRouter` me repo name ke sath `basename` attribute set karna (e.g. `basename="/DevConnect"`).
2. Default path (`path="/"`) par `<Navigate to="/marketplace" replace />` ya direct Marketplace component mount karna jisse website open hote hi Marketplace auto-load ho.

APP.JSX CONFIGURATION

```
// App.jsx - Adding basename for GitHub Pages & Default Landing Route
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Navbar from './components/Navbar';
import Marketplace from './pages/Marketplace';
import CartProvider from './context/CartContext';
import AuthProvider from './context/AuthContext';

function App() {
  return (
    <Router basename="/DevConnect">
      <AuthProvider>
        <CartProvider>
          <div style={{ minHeight: '100vh', display: 'flex', flexDirection: 'column' }}>
            <Navbar />
            <Routes>
              { /* Option A: Automatically redirect root URL to Marketplace */ }
              <Route path="/" element={ <Navigate to="/marketplace" replace /> } />

              { /* Option B: Direct root page load as Marketplace */ }
              <Route path="/marketplace" element={ <Marketplace /> } />

              { /* Other application routes */ }
            </Routes>
          </div>
        </CartProvider>
      </AuthProvider>
    </Router>
  );
}

export default App;
```

Key Rule: `basename` hamesha repository ke exact case-sensitive name ke equal hona chahiye (e.g. `/DevConnect`).

2. Removing Hardcoded Localhost & Centralizing Axios API Instance

Fixing CORS and 404/Connection Refused errors on production

Issue Encountered: Frontend code ke andar hardcoded `http://localhost:5000` URLs reh gaye the. Render ya live server par deploy hone ke baad client browser user ki local machine (localhost) pe request bhejta hai jo fail ho jata hai.

Step 2.1: Centralized Axios Instance Banayein (`src/api/axios.js`)

Har component me direct fetch/axios likhne ke bajaye environment-aware central Axios instance use karein:

`SRC/API/AXIOS.JS`

```
import axios from 'axios';

// Render backend live URL or Fallback to Environment Variable
const API_BASE_URL = import.meta.env.VITE_BACKEND_URL || "https://your-backend-service.onrender.com";

const api = axios.create({
  baseURL: API_BASE_URL,
  withCredentials: true, // Session cookies & headers support
  headers: {
    "Content-Type": "application/json"
  }
});

export default api;
```

Step 2.2: Payment Component Refactoring (Create Order)

Puraani `fetch("http://localhost:5000/...")` call ko clean Axios helper se replace karein:

`PAYMENT.JSX - ORDER CREATION`

```
// BEFORE (Hardcoded localhost - Breaks in Production):
- const response = await fetch("http://localhost:5000/api/payment/create-payment", { ... });

// AFTER (Production Ready with api instance):
+ import api from '../api/axios';

const handlePayment = async () => {
  try {
    const response = await api.post("/payment/create-payment", {
      orderId
    });

    const data = response.data;
    console.log("Payment Order Created Successfully:", data);

    // Initialize Razorpay Checkout Options
    initiateRazorpay(data);
  } catch (error) {
    console.error("Order creation failed:", error.response?.data || error.message);
  }
};
```

3. Razorpay Handler & Payment Verification Logic

Securing signature verification via Deployed API

Payment successful hone par Razorpay handler callback execute hota hai. Iske verification API call me bhi `api.post` use karna mandatory hai:

RAZORPAY MODAL HANDLER CALLBACK

```

const initiateRazorpay = (orderData) => {
  const options = {
    key: import.meta.env.VITE_RAZORPAY_KEY_ID || "rzp_test_YOUR_KEY",
    amount: orderData.amount,
    currency: orderData.currency,
    name: "DevConnect Store",
    description: "Order Checkout Transaction",
    order_id: orderData.id,
    handler: async function (response) {
      console.log("Razorpay Gateway Response:", response);
      try {
        // Calling deployed backend verification route
        const verifyResponse = await api.post("/payment/verify-payment", {
          paymentId: response.razorpay_payment_id,
          orderId: response.razorpay_order_id,
          signature: response.razorpay_signature
        });

        const verificationData = verifyResponse.data;
        console.log("Verification Result:", verificationData);

        if (verificationData.success) {
          clearCartState();
          navigate("/order-success");
        }
      } catch (err) {
        console.error("Payment Signature Verification Failed:", err);
        alert("Payment Verification Error. Please contact support.");
      }
    },
    theme: { color: "#0284c7" }
  };

  const razorpayInstance = new window.Razorpay(options);
  razorpayInstance.open();
};

```

4. Deployment Checklist & Terminal Execution Steps

Complete steps to test, build and push updates to GitHub Pages

Step	Action	Commands / Details
1	Global Search localhost	VS Code me <code>Ctrl + Shift + F</code> karein aur search karein <code>localhost</code> . Ensure karein koi component hardcoded URL use nahi kar raha.
2	Check package.json	Ensure scripts me <code>"predeploy": "npm run build"</code> aur <code>"deploy": "gh-pages -d dist"</code> configured hai.
3	Navigate to Client Folder	<code>cd client</code> (ya root frontend directory)
4	Execute Deploy Script	<code>npm run deploy</code> (Build generate hoke <code>gh-pages</code> branch par push ho jayega)

Step	Action	Commands / Details
5	Verify Live Site	2 minute wait karein, GitHub repo settings → Pages me status check karein aur link refresh karein (Ctrl + F5).

Result: Ab deployed website pe direct landing par Marketplace automatic show hoga, backend Render server ke sath correctly communicate karega, aur Razorpay payment verification seamlessly work karega.